SABLIER STAKING
Security Review



OCTOBER, 2025

www.chaindefenders.xyz
https://x.com/DefendersAudits

Lead Auditors



PeterSR



0x539.eth

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Commit Hash
  - Scope
  - Roles
- Executive Summary
  - Issues Found
  - Security Grade
- Findings
  - Low
  - Informational
  - Gas

Protocol Summary

Traditional staking protocols only support ERC20 tokens, leaving token stream holders unable to earn rewards on their vesting or payment streams. This protocol creates a significant opportunity cost for these users who decided to align themselves with the long term growth of the token. Sablier Staking Protocol is the first EVM protocol to enable staking of both ERC20 tokens and token streams

at the same time. This allows users to earn rewards on their vesting tokens without having to withdraw tokens from the streams or waiting for their streams to end.

Disclaimer

The Chain Defenders team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts.

Risk Classification

Likelihood/Impact	High	Medium	Low
High	H	H/M	M
Medium	H/M	M	M/L
Low	M	M/L	L

Audit Details

Commit Hash

Initial

2a1a9fd2a409244c0d96dba134b229fabee82646

Final

c9e9bdc803b8ccd80e2af1ef33e5ca5b29ece733

Scope

Id	Files in scope
1	SablierStakingState.sol
2	Errors.sol
3	Helpers.sol
4	DataTypes.sol
5	SablierStaking.sol

Roles

Id	Roles
1	Comptroller
2	Pool Admin
3	Staker

Executive Summary

Issues Found

Severity	Count	Description
High	0	Critical vulnerabilities
Medium	0	Significant risks
Low	2	Minor issues with low impact
Informational	2	Best practices or suggestions
Gas	1	Optimization opportunities

Security Grade



Based on the audit findings and code quality, the overall protocol security grade is 98.00 / 100.00. This reflects a high security posture with small room for improvement.

Findings

Low

Low 01 Decimal Mismatch And Rounding Issues Due To No Whitelisting Of Staking And Reward Tokens

Location

SablierStaking.sol

SablierStakingState.sol

Description

The `SablierStaking` :: `createPool` function allows any user to create a staking pool by specifying various parameters, including the `rewardToken` and the `stakingToken`. The contract currently does not enforce any checks on token addresses, meaning any ERC20 token can be designated as a reward or staking token.

This creates two related risks:

1. Decimals mismatch / rounding issues - The reward distribution arithmetic assumes a single consistent numeric scale. If staking and reward tokens have very different decimals (e.g. staking token 24 decimals such as YAM-V2, reward token 2 decimals such as Gemini USD), per-token reward values can become so small that integer division truncates to zero. This results in legitimate reward rounds where `claimRewards` computes `0` claimable reward for users, causing reverts and a broken user experience.
2. Reward-token scams - A malicious actor could create a pool that uses a legitimate and valuable token for staking but offers a fake or worthless token as the reward. Unsuspecting users may pay gas/fee to claim valueless tokens and/or be unable to meaningfully trade them.

Proof of Concept

1. Add the following line to the `claimRewards.t.sol` file:

```
import { ERC20Mock } from "@sablier/evm-utils/src/mocks/erc20/ERC20Mock.sol";
```

2. Add the following test to the `claimRewards.t.sol` file:

```
function test_DifferentDecimalsTokensPool() external whenNoDelegateCall whenMinFeePaid whenFeeOnRewardsNotTooHigh {
    // --- Arrange: create a fresh pool with a tiny reward and long period ---
    setMsgSender(users.poolCreator);

    ERC20Mock rewardTokenWithDifferentDecimals = new ERC20Mock("Reward Token", "REWARD_TOKEN", 2);
    ERC20Mock stakingTokenWithDifferentDecimals = new ERC20Mock("Staking Token", "STAKING_TOKEN", 24);

    uint40 start = uint40(block.timestamp + 1); // valid start in future
    uint40 end = start + 10_000_000; // long reward period
```

```
10      uint128 tinyReward = 100 * 10e2; // 100 tokensRewads
11
12      // Approve and create pool (transfers tinyReward from admin to
        contract).
13      deal({ token: address(rewardTokenWithDifferentDecimals), to: users
        .poolCreator, give: tinyReward });
14      rewardTokenWithDifferentDecimals.approve(address(sablierStaking),
        tinyReward);
15      uint256 newPoolId = sablierStaking.createPool(
16          users.poolCreator,
17          stakingTokenWithDifferentDecimals,
18          start,
19          end,
20          rewardTokenWithDifferentDecimals,
21          tinyReward
22      );
23
24      // --- Arrange: user stakes a large amount so per-token share is
        tiny ---
25      uint128 stake = 50 * 10e24; // 50 tokensStaked
26
27      deal({ token: address(stakingTokenWithDifferentDecimals), to:
        users.recipient, give: stake });
28      deal({ token: address(stakingTokenWithDifferentDecimals), to:
        users.sender, give: stake });
29
30      setMsgSender(users.recipient);
31      stakingTokenWithDifferentDecimals.approve(address(sablierStaking),
         stake);
32      sablierStaking.stakeERC20Token(newPoolId, stake);
33
34      setMsgSender(users.sender);
35      stakingTokenWithDifferentDecimals.approve(address(sablierStaking),
         stake);
36      sablierStaking.stakeERC20Token(newPoolId, stake);
37
38      // --- Act: warp slightly into the reward period so a small slice
        is distributable ---
39      vm.warp(end + 1);
40
41      // --- Assert: due to integer division/rounding, computed rewards
        == 0 -> revert expected ---
42      setMsgSender(users.recipient);
43      vm.expectRevert(
44          abi.encodeWithSelector(Errors.
        SablierStaking_ZeroClaimableRewards.selector, newPoolId, users.
        recipient)
```

```
45      );
46
47      sablierStaking.claimRewards{ value: STAKING_MIN_FEE_WEI }(
        newPoolId, FEE_ON_REWARDS);
48
49      setMsgSender(users.sender);
50      vm.expectRevert(
51          abi.encodeWithSelector(Errors.
        SablierStaking_ZeroClaimableRewards.selector, newPoolId, users.
        sender)
52      );
53
54      sablierStaking.claimRewards{ value: STAKING_MIN_FEE_WEI }(
        newPoolId, FEE_ON_REWARDS);
55  }
```

Recommendation

To reduce risk and improve correctness, implement the following:

1. Whitelist tokens (both reward and staking tokens)
    - Add two whitelists: `_whitelistedRewardTokens` and `_whitelistedStakingTokens`.
    - Only the `comptroller` (trusted authority) may add/remove tokens from both whitelists.
    - `createPool` must require that both `stakingToken` and `rewardToken` are whitelisted.
2. Enforce or normalize decimals
    - Option A (strict, simple): require the staking token and reward token to have the same `decimals()`; reject pools where decimals differ. This is the easiest to get correct and prevents rounding bugs.
    - Option B (robust): normalize arithmetic to a common internal scale (e.g. 18 decimals) using `decimals()` of each token. Convert `rewardAmount` and per-token rates into that internal scale before dividing. This is more flexible but requires careful implementation and thorough tests (and handling of tokens that don't implement `decimals()`).

At minimum, add an explicit check/documentation and a unit test demonstrating different-decimals behavior.

Status

Acknowledged

Response: Given the self-custodial nature of the protocol, the recommendations cannot be implemented. However, as a response to malicious pools, relevant warnings will be displayed to the users.

Low 02 Improper Reward Distribution Logic In `configureNextRound` And `createPool`

Location

SablierStaking.sol:222

SablierStaking.sol:294

Description

The functions `SablierStaking::configureNextRound` and `SablierStaking::createPool` allow pool admins to configure a new reward round by specifying its start time, end time, and reward amount.

However, the current logic contains flaws that can cause rewards to become permanently locked within the contract:

1. Excessive Duration vs. Rewards

If `(endTime - startTime) > rewardAmount`, then sometimes all rewards will remain stuck in the contract.

2. Non-Divisible Rewards

If `rewardAmount % (endTime - startTime) > 0`, then the remainder (`x`) rewards will be stuck and undistributed.

This creates scenarios where either some or all rewards are inaccessible.

Proof of Concept

Add this test to `claimRewards.t.sol`:

```
function test_RoundDownResultsInZeroRewards_CreateNewPool()
    external
    whenNoDelegateCall
    whenMinFeePaid
    whenFeeOnRewardsNotTooHigh
{
    // --- Arrange: create a fresh pool with a tiny reward and long period ---
    setMsgSender(users.poolCreator);

    uint40 start = uint40(block.timestamp + 1);    // valid start in future
    uint40 end   = start + 10_000_000;                // long reward period
    uint128 tinyReward = 1;                          // intentionally tiny to force integer rounding

    // Approve and create pool (transfers tinyReward from admin to contract).
    rewardToken.approve(address(sablierStaking), tinyReward);
    uint256 newPoolId = sablierStaking.createPool(users.poolCreator, stakingToken, start, end, rewardToken, tinyReward);

    // --- Arrange: user stakes a large amount so per-token share is tiny ---
    uint128 largeStake = uint128(1 ether);

    deal({ token: address(stakingToken), to: users.recipient, give: largeStake });
    deal({ token: address(stakingToken), to: users.sender, give: largeStake });

    setMsgSender(users.recipient);
    stakingToken.approve(address(sablierStaking), largeStake);
    sablierStaking.stakeERC20Token(newPoolId, largeStake);

    setMsgSender(users.sender);
    stakingToken.approve(address(sablierStaking), largeStake);
    sablierStaking.stakeERC20Token(newPoolId, largeStake);

    // --- Act: warp after the reward period end ---
    vm.warp(end + 1);

    // --- Assert: due to integer division/rounding, computed rewards == 0 -> revert expected ---
    setMsgSender(users.recipient);
    vm.expectRevert(
```

```
38          abi.encodeWithSelector(Errors.
    SablierStaking_ZeroClaimableRewards.selector, newPoolId, users.
    recipient)
39      );
40
41      sablierStaking.claimRewards{ value: STAKING_MIN_FEE_WEI }(
    newPoolId, FEE_ON_REWARDS);
42
43      setMsgSender(users.sender);
44      vm.expectRevert(
45          abi.encodeWithSelector(Errors.
    SablierStaking_ZeroClaimableRewards.selector, newPoolId, users.
    sender)
46      );
47
48      sablierStaking.claimRewards{ value: STAKING_MIN_FEE_WEI }(
    newPoolId, FEE_ON_REWARDS);
49  }
```

Recommendation

Introduce a validation function to enforce reward distribution constraints. This should be used in both `createPool` and `configureNextRound`:

```
function checkRewardAmount(
    uint256 startTime,
    uint256 endTime,
    uint256 rewardAmount
) {
    if (endTime - startTime > rewardAmount ||
        rewardAmount % (endTime - startTime) ≠ 0)
    {
        revert Errors.
    SablierStaking_RewardAmountWillBeNotFullyDistributed();
    }
}
```

Status

Acknowledged

Response: Given the design of the rewards calculation, there will always be some dust left in the pool as a result of rounding errors introduced by the divisions. The validation function recommended in the report would offer a very bad UX for pool creators as they won't be able to select reward amounts as a multiple of

1e18. There may always be some dust left in the pool but the dust amount will be very insignificant in practice.

Informational

Info 01 Excess Fee Not Refunded

Location

SablierStaking.sol:149

Description

The `SablierStaking` :: `claimRewards` function requires a fee payment in the native asset to be sent along with the transaction (`msg.value`). The function calculates the required minimum fee, `minFeeWei`, and verifies that the sent `msg.value` is at least this amount.

However, if a user sends an amount greater than `minFeeWei`, the entire `msg.value` is transferred to the `comptroller` contract. The excess amount (`msg.value - minFeeWei`) is not refunded to the user, leading to a potential loss of funds for the user if they overpay the fee.

Recommendation

It is recommended to modify the `claimRewards` function to only forward the required `minFeeWei` to the `comptroller` and refund any excess amount to the `msg.sender`.

A gas-efficient approach would be to only transfer `minFeeWei` to the comptroller. If `msg.value` is greater than `minFeeWei`, the excess ETH will remain in the contract. A separate withdrawal function could be implemented for users to reclaim their excess payments.

Status

Acknowledged

Response: Business decision.

Info 02 Excessive Reward Round Duration Can Block Future Distributions

Location

SablierStaking.sol:222

SablierStaking.sol:294

Description

The `createPool` and `configureNextRound` functions are used to set the start and end times for reward distribution periods. The contract logic in `configureNextRound` requires that a new reward round can only be started after the current one has concluded (`pool.endTime < block.timestamp`).

However, neither function imposes an upper limit on the duration of a reward period (`endTime - startTime`). This absence of a check allows a pool administrator to set an `endTime` far in the future, either by mistake (e.g., a typo in the timestamp) or intentionally. If an excessively long reward period is set, the pool becomes effectively locked. The administrator will be unable to configure a new reward round until this distant `endTime` is reached, rendering the pool unusable for future distributions for an extended period.

Recommendation

It is recommended to introduce a configurable maximum duration for reward periods to prevent pools from being inadvertently locked. This can be achieved by adding a state variable, such as `maxRewardPeriod`, which can be managed by a privileged account like the `comptroller`.

Both the `createPool` and `configureNextRound` functions should then be updated to validate that the duration of the new reward period does not exceed this maximum limit.

```
// ... existing code ...

contract SablierStaking is

// ... existing code ...

```

```
7       /// @notice The maximum duration of a reward period in seconds.
8       uint40 public maxRewardPeriod;
9
10  //  ... existing code ...
11
12      constructor(address initialComptroller) Comptrollerable(
    initialComptroller) {
13          // Effect: Set the next pool ID to 1.
14          nextPoolId = 1;
15          // Effect: Set the default max reward period to 1 year.
16          maxRewardPeriod = 365 days;
17      }
18
19  //  ... existing code ...
20
21      /// @notice Sets a new maximum reward period.
22      /// @param newMaxRewardPeriod The new maximum reward period in
    seconds.
23      function setMaxRewardPeriod(uint40 newMaxRewardPeriod) external
    onlyComptroller {
24          maxRewardPeriod = newMaxRewardPeriod;
25      }
26
27  //  ... existing code ...
28
29      function configureNextRound(
30
31  //  ... existing code ...
32
33          // Check: the reward period does not exceed the maximum
    allowed duration.
34          if (newEndTime - newStartTime > maxRewardPeriod) {
35              revert Errors.SablierStaking_RewardPeriodTooLong(
    newEndTime - newStartTime, maxRewardPeriod);
36          }
37
38  //  ... existing code ...
39
40      function createPool(
41
42  //  ... existing code ...
43
44          // Check: the reward period does not exceed the maximum
    allowed duration.
45          if (endTime - startTime > maxRewardPeriod) {
46              revert Errors.SablierStaking_RewardPeriodTooLong(endTime -
     startTime, maxRewardPeriod);
```

```
47          }
48
49  //  ... existing code ...
```

Status

Acknowledged

Response: Adding `maxRewardDuration` would limit the ability of pool creators to configure pools as per their requirements, which is something that does not align with protocol use case.

Gas

Gas 01 Inefficient Rewards Snapshot Calculation

Location

SablierStaking.sol:660

Description

The `SablierStaking::_rewardsDistributedSinceLastSnapshot` function repeatedly reads fields from storage (`_pools[poolId]`) and uses small-width types (`uint40`, `uint128`) which are then repeatedly promoted to `uint256` for comparisons and arithmetic. Concretely the code makes multiple independent SLOADs for `totalStakedAmount`, `endTime`, `startTime`, `snapshotTime`, and `rewardAmount` (each `_pools[poolId].<field>`), and performs multiple implicit casts between `uint40`/`uint128` and `uint256`. Each SLOAD is relatively expensive compared with stack/local operations and each implicit cast adds tiny costs; together these cause a material gas overhead every time the function runs.

Because this helper function will be called frequently, the overhead is not just academic: the repeated storage reads and repeated casts add up.

Recommendation

Do the following steps:

1. Cache storage once and work in locals - read `Pool storage p = _pools[poolId];` and copy only the needed fields into local `uint256` variables (e.g. `startT`, `endT`, `snapT`, `rewardAmount`, `totalStaked`). Locals use the stack and are far cheaper than repeated SLOADs.
2. Do casts once - convert `uint40`/`uint128` to `uint256` once when loading into locals instead of letting the compiler insert many implicit promotions during comparisons and arithmetic.
3. Early cheap short-circuits — place cheap checks early to avoid unnecessary reads/compute in common cases.
4. Avoid repeated work / redundant checks — compute `startingTimestamp` / `endingTimestamp` with single ternary expressions.

Applying these recommendations is a no-risk change with clear gas benefit. After all changes the function will look like this:

```
function _rewardsDistributedSinceLastSnapshot(uint256 poolId) private view returns (uint256 rewardsDistributed) {
    // Read storage once (cheap pointer), then copy only needed fields into locals.
    Pool storage p = _pools[poolId];

    // Read only the fields we need into stack variables (uint256) to avoid repeated SLOADs
    // and repeated small-type casts during arithmetic.
    uint256 totalStaked = uint256(p.totalStakedAmount);
    if (totalStaked == 0) return 0;

    uint256 startT = uint256(p.startTime);
    uint256 endT = uint256(p.endTime);
    uint256 snapT = uint256(p.snapshotTime);

    uint256 nowT = block.timestamp;
    // If the start time is in the future, nothing distributed yet.
    if (nowT < startT) return 0;

    // If snapshot is at or after pool end, nothing new to distribute.
    if (snapT ≥ endT) return 0;

    // Compute actual range [startingTimestamp, endingTimestamp]
    uint256 startingTimestamp = snapT ⩽ startT ? startT : snapT;
    uint256 endingTimestamp  = endT ⩽ nowT ? endT : nowT;

    // If ending is not after starting then 0
    if (endingTimestamp ⩽ startingTimestamp) return 0;

```

```
28      // rewardAmount and rewardsPeriod as uint256 for clean arithmetic
29      uint256 rewardAmount = uint256(p.rewardAmount);
30      uint256 rewardsPeriod = endT - startT;
31
32      unchecked {
33          uint256 elapsedTime = endingTimestamp - startingTimestamp;
34
35          // Compute proportional reward
36          return (rewardAmount * elapsedTime) / rewardsPeriod;
37      }
38  }
```

Status

Fixed in PR 45